

COMP90077

Lecture 6

Bipartite Matchings and
Linear Programming I

William Umboh
University of Melbourne

Caution

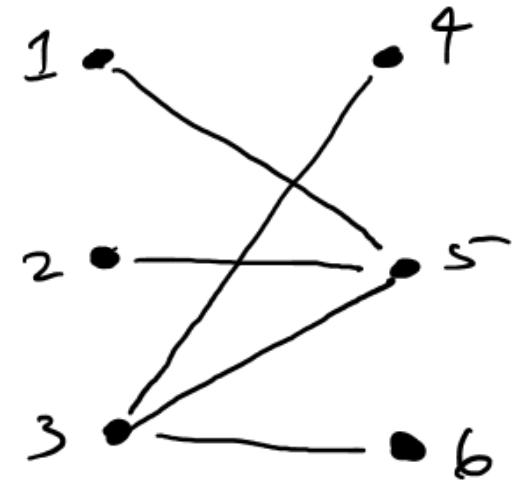
- Slides are designed as presentation aid to go along with lecture, not as study material
- Slides not designed to make sense on their own

Today

- Last time: 2-approximation for maximum matching and minimum vertex cover in **general graphs**
- Exact algorithm for both in **bipartite graphs**
- Linear programming

Recap

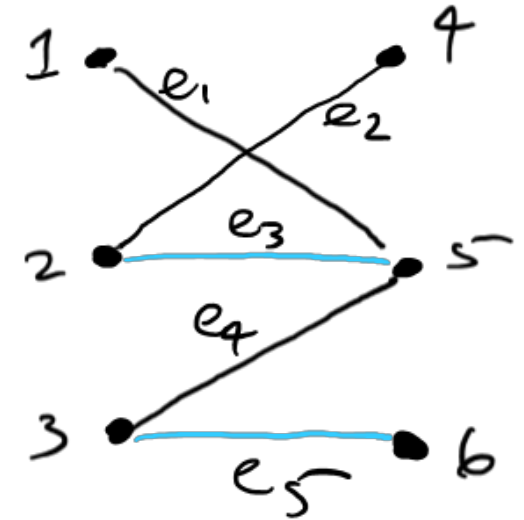
- Bipartite graph $G = (L \cup R, E)$
 - Edges can only go between left part and right part
- Matching M
 - Set of edges
 - Every vertex incident to at most one edge in M
 - Example: $\{(2,5), (3,6)\}$
- Vertex cover C
 - Set of vertices
 - Every edge incident to at least one vertex in C
 - Example: $\{5, 3\}$
- Duality Lemma: For every matching M and vertex cover C , we have $|M| \leq |C|$.
- Quiz: Maximum matching in this example?



Augmenting Paths

With respect to a matching M ,

- A vertex is *free* if it is not incident to M
 - Example: 1 and 4 are free
- A path is *alternating* if it alternates between edges in M and not in M
 - Example: e_4, e_3, e_2
 - Example: e_3, e_4, e_5
- An alternating path is *augmenting* if it starts and ends with free vertices, i.e. it starts and ends with edges not in M
 - Are above paths augmenting?
 - Example: e_1, e_3, e_2





Augmenting Paths Gonna Augment

An alternating path is *augmenting* if it starts and ends with free vertices, i.e. it starts and ends with edges not in M

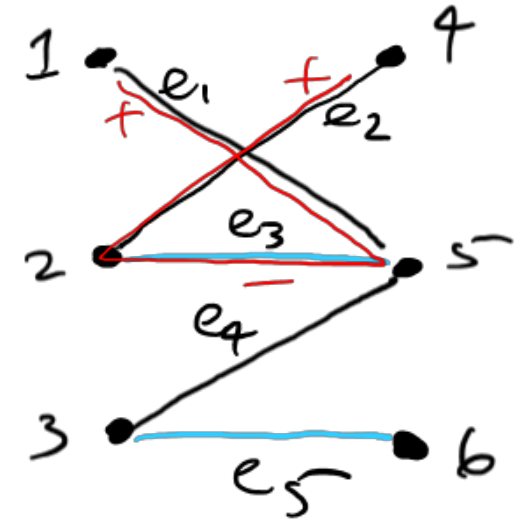
Given matching M and an augmenting path P , define their symmetric difference (aka XOR)

$$M \oplus P = \underbrace{(M \setminus P)} \cup \underbrace{(P \setminus M)}$$

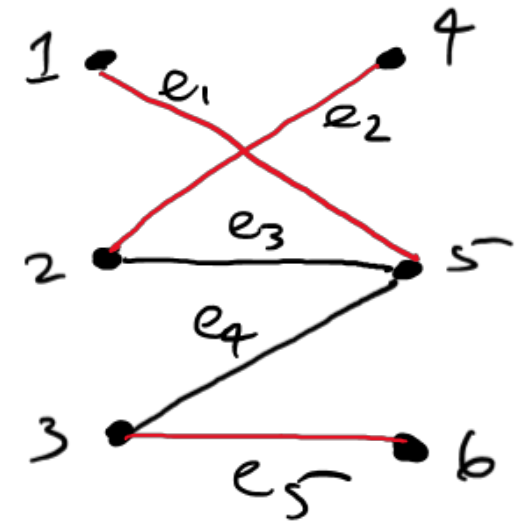
i.e. take M , replace common edges with new edges from P .

Lemma. $M \oplus P$ is matching of size $|M| + 1$.

Pf Sketch. $|M \setminus P| = |P \setminus M| - 1$



M



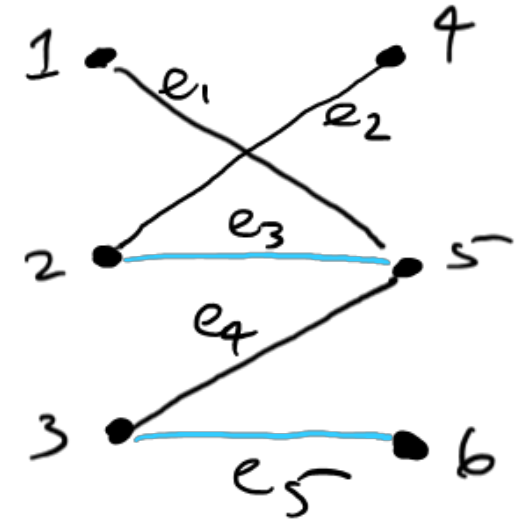
$M \oplus P$

Bipartite Matching Algorithm

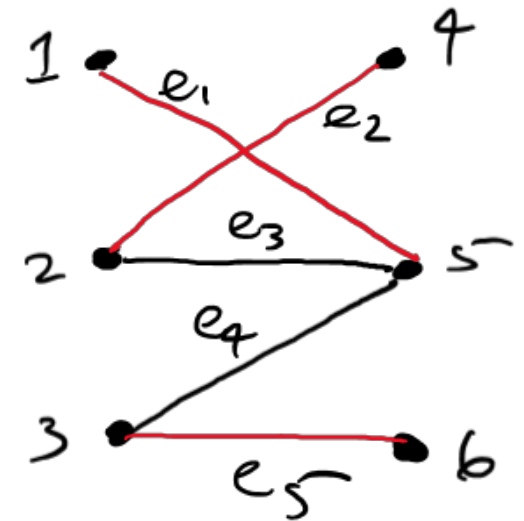
- Initialise $M = \emptyset$
- while there exists an augmenting path P :
 - $M = M \oplus P$
- return M

Remains to show:

1. If no augmenting path, then M is maximum
2. How to find augmenting path, if it exists



M



$M \oplus P$

Characterization of Optimality

Thm. M is maximum if and only if there is no augmenting path.

Characterization of Optimality

Thm. M is not maximum if and only if there is ^{an} ~~no~~ augmenting path.



Claim If M is not maximum, then there is an augmenting path.

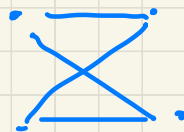
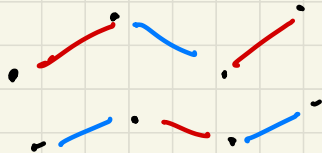
Pf Suppose M^* is max matching and $|M^*| > |M|$.

Consider $M^* \oplus M = (M^* \setminus M) \cup (M \setminus M^*)$

Max degree of $M^* \oplus M$ is at most 2.

$\Rightarrow$ decompose into vertex-disjoint^v cycles and paths.
even-length

M^* —
 M —



M^* and M have equal # of edges in even-length cycles and paths.

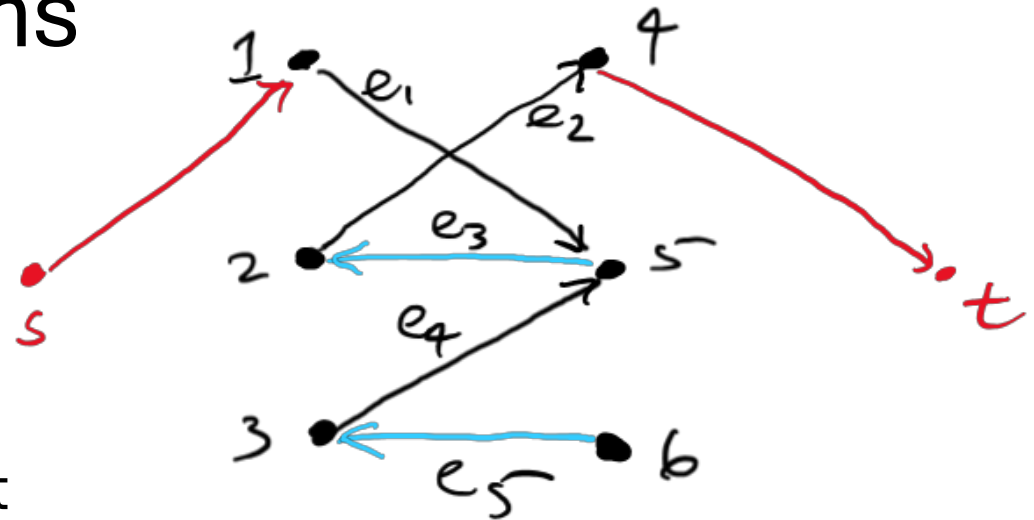
Since $|M^*| > |M|$, there exists a path starts and ends w/ M^*

Finding Augmenting Paths

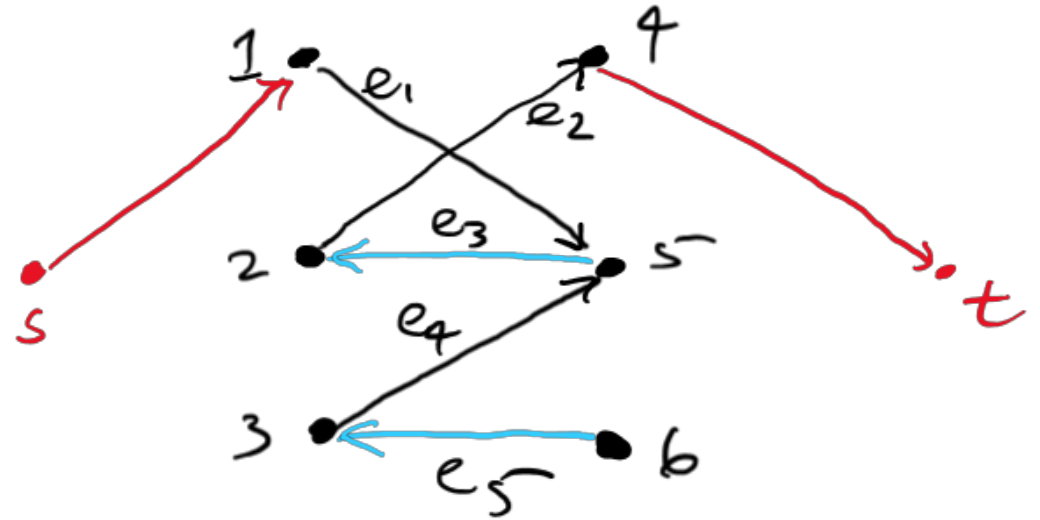
Residual Network

- Directed graph
- Direct each edge in M right to left
- Direct each edge not in M left to right
- Add vertex s and directed edges from s to each free vertex on L
- Add vertex t and directed edges to t from each free vertex on R

Augmenting path = $s - t$ path in residual network



Checking Optimality



- Finding a maximum matching can take up to $n/2$ augmenting path computations, where n is number of vertices
- Checking optimality takes only 1 augmenting path computation!

Vertex Cover

Thm. (Strong Duality)

In bipartite graphs, minimum vertex cover = maximum matching.

Then let M^* be a max matching.

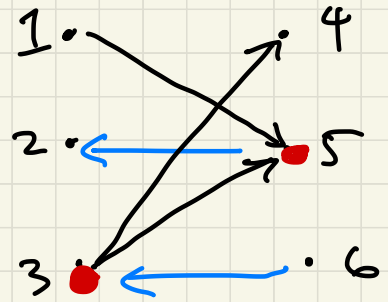
L^* = left endpoints of M^*

R^* = right endpoints of M^*

$Q = \{v : v \text{ is reachable from a free vertex of } L \text{ via an alternating path}\}.$

Then, $C^* = (L^* \cap Q) \cup (R^* \cap Q)$

is a min vertex cover.



$$L^* = \{2, 3\}$$

$$R^* = \{5, 6\}$$

$$Q = \{1, 5, 2\}.$$

$$C^* = \{3, 5\}.$$

Then Let M^* be a max matching.

L^* = left endpoints of M^*

R^* = right endpoints of M^*

$Q = \{v : v \text{ is reachable from a free vertex of } L \text{ via an alternating path}\}.$

Then, $C^* = (L^* \setminus Q) \cup (R^* \cap Q)$

is a min vertex cover.

Claim C^* vertex cover

pf Let $e = (u, v)$ $u \in L, v \in R$

Cases ① $u \in L \setminus L^*, v \in R \setminus R^* \Rightarrow M^*$ not max

② $u \in L \setminus L^*, v \in R^* \Rightarrow v \in Q$

③ $u \in L^*, v \in R \setminus R^* \Rightarrow u \in L^* \setminus Q$

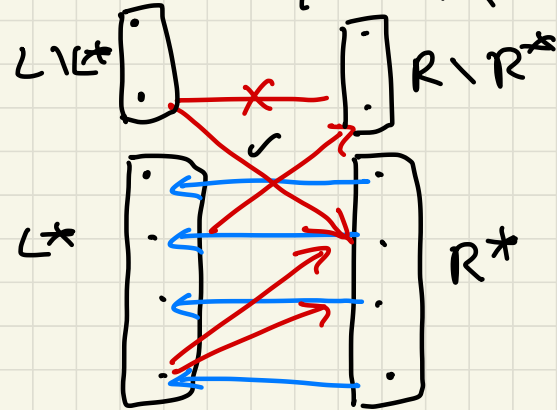
④ $u \in L^*, v \in R^*$
 $(u, v) \notin M^*,$

$u \in Q \Rightarrow v \in Q$

⑤ $(u, v) \in M^*$

$v \notin Q \Rightarrow u \in L^* \setminus Q$

$v \in Q \Rightarrow u \notin L^* \setminus Q$



Linear Programming, or: How I Learnt to Stop Worrying and Relax (Fractionally)

Vertex Cover

$$\min |C|$$

$$\text{s.t. } C \subseteq V$$

$$|C \cap \{u, v\}| \geq 1 \quad \forall (u, v) \in E$$

Choose subset of vertices

vertex cover

Linear Program

$$\min \sum_v x_v$$

$$\text{s.t. } x_u + x_v \geq 1 \quad \forall (u, v) \in E$$

$$x_v \in \{0, 1\} \quad \forall v$$

For each vertex, variable x_v

$$x_v = \begin{cases} 1 & \text{if } v \in C \\ 0 & \text{if } v \notin C \end{cases}$$

$\longleftrightarrow$ x feasible

Set Cover

$$\min |I|$$

$$\text{s.t. } I \subseteq [m]$$

$$|\{i \in I : s_i \ni e\}| \geq 1 \quad \forall e \in U$$

Linear Program

$$\min \sum_i x_i$$

$$\text{s.t. } \sum_{i \in I : s_i \ni e} x_i \geq 1 \quad \forall e \in U$$

Variables x_i

$$x_i = \begin{cases} 1 & \text{if } i \in I \\ 0 & \text{o/w} \end{cases}$$

Matching

$$x_e = \begin{cases} 1 & \text{if } e \in M \\ 0 & \text{if } e \notin M \end{cases}$$

$$\max \sum_e x_e$$

$$\text{s.t. } \sum_{\substack{e \text{ incident} \\ \text{to } v}} x_e \leq 1 \quad \forall v$$

$$x_e \in \{0, 1\}.$$

Knapsack

Items weight w_i
size s_i

Capacity C

$$x_i = \begin{cases} 1 & \text{if } i \in S \\ 0 & \text{if } i \notin S \end{cases}$$

$$\max \sum_i w_i x_i$$

$$\text{s.t. } \sum_i s_i x_i \leq C$$

$$x_i \leq 1$$

Linear Program

n variables, m constraints

Coefficients: $[a_1, \dots, a_n] = a$

$$\begin{array}{|c|} \hline b_{1,1}, b_{1,2}, b_{1,3}, \dots, b_{1,n} \\ \hline \vdots \\ \hline b_{m,1}, b_{m,2}, \dots, b_{m,n} \\ \hline \end{array} \quad \begin{array}{|c|} \hline c_1 \\ \hline \vdots \\ \hline c_m \\ \hline \end{array}$$

$B \qquad c$

Variable vector x

Integer Linear Program

Linear Program

Thm. $\exists$ $\text{poly}(m, n)$ -time algorithm for LPs

$$\max \sum_i a_i \cdot x_i$$

$$\text{s.t. } b_{1,1}x_1 + \dots + b_{1,n}x_n \leq c_1$$

$\vdots$

$$\max \quad a^T x$$

$$\text{s.t. } Bx \leq c$$

$$x \in \{0, 1\}^n$$

$$x \in [0, 1]^n$$